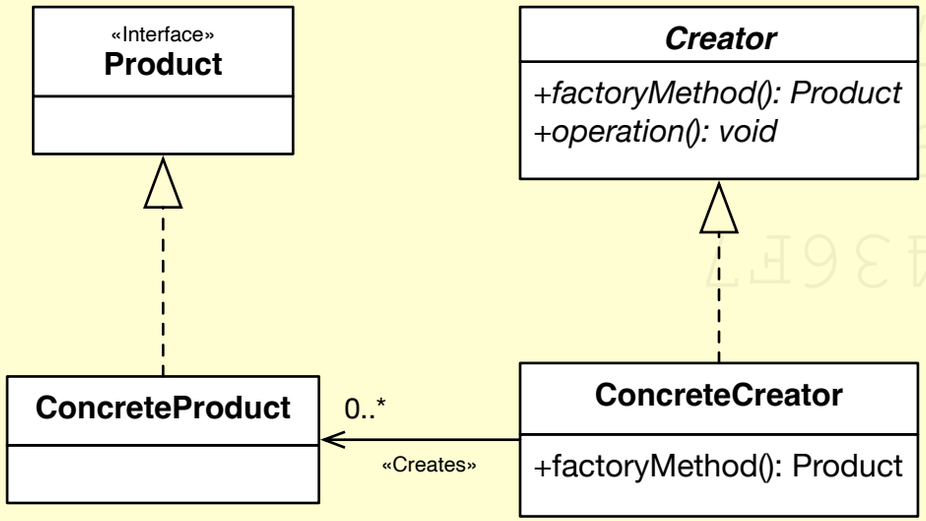
Encapsulate Object Creation with Singletons, Factories & Builders (Part I)

Creational design pattens are concerned with the **best way of instantiating objects**. The intent is to **abstract and encapsulate how objects are created** and arranged and to decouple the user of a class from dependencies on concrete types. The use of the ***new*** keyword is essentially hard coding and creates a permanent dependency that reduces the flexibility and reuse of software.

Factory - One Shot Object Creation

Define an interface for creating an object, but let subclasses decide which class to instantiate. A factory method lets a class defer instantiation to subclasses.

A factory **encapsulates the instantiation of concrete types** and decouples the user of a class from its implementation. A factory method **returns an instance of one of several possible classes**, depending on the data provided to it. This enables us to vary the implementation without affecting dependencies from other classes. Typically, all classes returned by the factory share **methods defined by a common super type**.



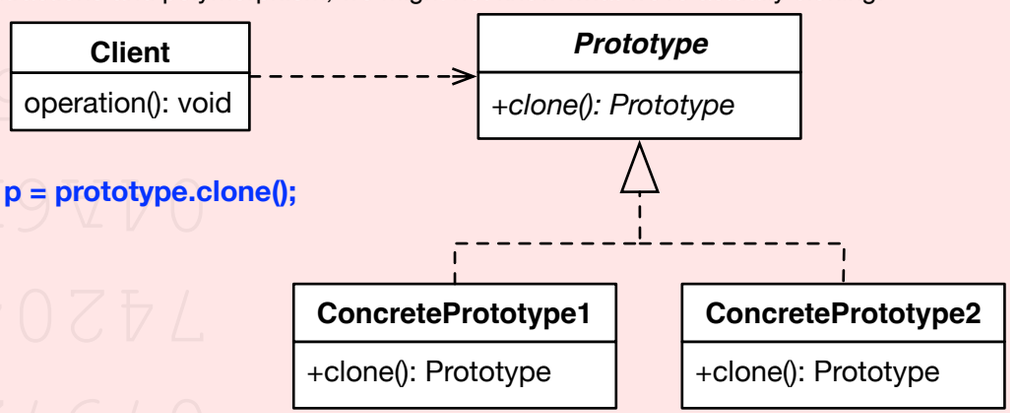
Creator contains implementations for all of the methods to manipulate instances of *Product*, except for the factory method. *ConcreteCreator* is responsible for creating one or more concrete products and is the only class that has knowledge of **how to create products**. Each subtype of *Creator* performs a task differently and is optimised for different kinds of data. The factory method and the creator do not have to be abstract and the *Creator* and *ConcreteCreator* are often merged together and combined with a singleton in one class. Such a class is called a **Singleton Factory**.

★★★ The factory pattern is actually an implementation of the ***Dependency Inversion Principle*** and can be recognised by the use of **creational methods** that return implementations of interfaces.

Prototype - Copy That

Specify the kinds of objects to create using a prototypical instance and create new objects by copying this prototype.

The prototype pattern is used when **creating a class instance that is either expensive or complicated**. The basic idea is to **use an object parameter as a template** for creating a new object. In Java, this is done by implementing the marker interface *Cloneable* and then over-riding the *protected Object clone() throws CloneNotSupportedException* method. The implementation details of the prototype object may be unknown and **hidden by encapsulation**. When done using abstractions and polymorphism, we might not know what we're actually cloning.

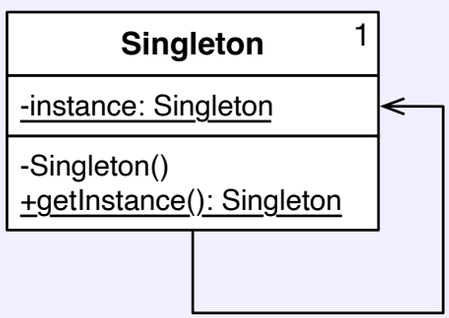


The clone created from a prototype is typically a **shallow copy**, i.e. all state from the original is copied at a bit level, including object references. Any composed or referenced types will have to be cloned in turn to create a **deep copy**.

Singleton - One of a Kind

*The singleton pattern ensures that **a class has only one instance**, and provides a global point of access to it.*

A singleton manages the instance of itself and access to it is through the class itself. Access is controlled by declaring a **private constructor** and then using a **static public getInstance() method** to create a single instance. The singleton pattern upholds ***Single Responsibility Principle*** and the **Highlander Principle** ("*There can be only one*"). Singletons can be declared **eagerly** as an instantiated class field, or **lazily** when the *getInstance()* method is called for the first time. Singletons can be difficult to test and can cause problems in multithreaded applications. The *getInstance()* method can be **synchronized** to force threads to wait for access to the method but this can cause a bottleneck. Eager singletons can be protected using a **double-checked lock and** by applying the **volatile** keyword.



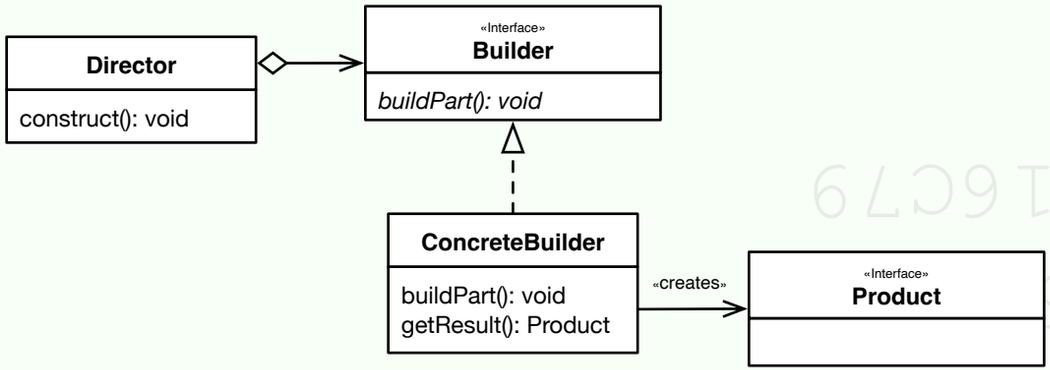
★★★ A singleton can be recognised by the use of a **creational class method** like *getInstance*() that **returns an instance of itself**.

Encapsulate Object Creation with Singletons, Factories & Builders (Part II)

Builder - Create a Complex Object Step by Step

*Separate the **construction of a complex object from its representation** so that the same construction process can create different representations.*

A builder allows us to choose to create the type of object we want but keeps the construction process the same. Builders focus on **constructing a complex object in a multistep varying process** and then encapsulate how the composite object is created.



The *Director* knows what needs to be built and knows the set of steps required to build it, but doesn't know how to put individual components together. The interface *Builder* abstracts the operations for creating an instance of *Product*. Subtypes of *Builder* know how to assemble different kinds of components for different kinds of products, but don't know what it is that they are building. A **client configures a *Director*** with an instance of *Builder* to create a complex object. In practice, the *Director* is often not used at all and the *Builder* may be concrete (**a class abstraction**) instead of an interface (**a role abstraction**), e.g. *StringBuilder* and *DocumentBuilder*. If the *Director* or *Builder* require methods to be called in a particular sequence, we may end up accidentally inducing **sequential coupling** (an anti-pattern) into a design.

Fluent Interfaces

A builder API can be greatly simplified by using a fluent interface. A fluent interface is designed to **make the configuration of complex value objects readable and to flow naturally** by moving object construction behind a thoughtful interface. The idea is to create an API similar to a **Domain-Specific Language (DSL)** such as SQL, CSS and the **Java Stream API**. Creating a fluent interface requires greater design effort and thought. One of the side effects of a fluent interface is the use of **setters methods that return a value**, usually of the interface itself.

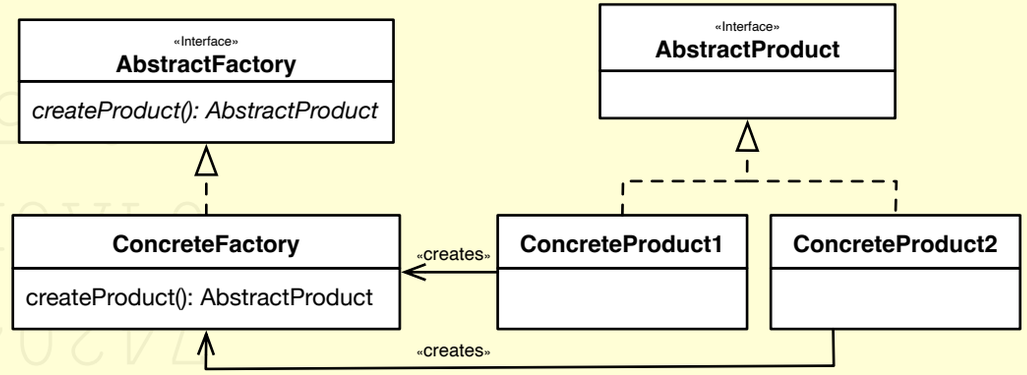
```
//Create a shopping trolley with a fluent builder. Construction should “flow naturally”.
Order order = new OrderBuilder("IE-123")
  .with("978-1447269045", "Seek the Fair Land", 2, 22.99f)
  .with("978-0330303286", "The Silent People", 1, 22.99f)
  .with("978-0330303262", "The Scorching Wind", 1, 55.85f)
  .with("978-0330397872", "Flight of the Doves", 4, 26.47f)
  .with("978-1447269205", "Rain on the Wind", 4, 22.99f)
  .with(new Address("Walter Macken Road", "Mervue", County.Galway, "AAA1915"))
  .build(); //Complete the construction and return an instance of the interface Order
```

★★★ A builder can be recognised by the use of **creational or fluent methods** that return an instance of a class or role abstraction.

Abstract Factory - A Factory of Factories

*Provide an interface for **creating families of related or dependent objects** without specifying their concrete classes.*

An abstract factory enables a client to use an abstract interface to **create a set of related products** without knowing about the concrete products that are actually produced. The client is decoupled from any of the specifics of the concrete products. An abstract factory returns one of several factories and is **a level of abstraction higher than the Factory pattern**. It can be thought of as **a factory of factories**.



AbstractFactory defines an interface that each *ConcreteFactory* implements, i.e. a set of methods for producing instances of *AbstractProduct*. *ConcreteFactory* types implement different product families and a client uses one of these factories to avoid instantiating a product object directly. An abstract factory **isolates** and **encapsulates** the **concrete classes** that it is responsible for generating and a client is not required to know about or depend on them. This decoupling enables the changing of different *ConcreteFactory* and *ConcreteProduct* implementations without adversely affecting consumers of the classes. While derived types can implement addition methods to those declared in a base types, these need to be type-checked as cast before they can be used, e.g.

```
if (obj instanceOf ConcreteProduct cp){  //Avoid tight coupling where possible
  //Do something with Java 15 “matched” cp
  cp.executeAdditionalOperation();
}
```

The **JAXP** API uses an abstract factory to create a DOM node tree:

```
//The abstract factory method newInstance() creates a new factory instance.
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
//The factory method newDocumentBuilder() creates a builder
DocumentBuilder db = dbf.newDocumentBuilder();
//The builder then creates a complex object for us, a DOM node tree
Document doc = db.parse("....");
```

★★★ An abstract factory can be recognised by the use of **creational methods** like *newInstance*() that **return the abstract factory itself** which is then used in turn to **create an another abstraction** through a factory method.